Cum integrez spotify??

A. Spotify Web API (REST, over HTTP)

Ce poți face cu el:

- **Ce ascultă utilizatorul acum**
  - Endpoint: “Get Currently Playing Track” – îți dă piesa curentă, timestamp, progress, context etc. Spotify for Developers
  - Ai nevoie de scope: `user-read-currently-playing` / `user-read-playback-state`. Spotify for Developers
- **Top artiști / piese ale utilizatorului**
  - Endpoint: “Get User's Top Artists and Tracks” – /me/top/{type}, unde type = `artists` sau `tracks`. Spotify for Developers+1
  - Scope: `user-top-read`. Spotify for Developers
- **Alte chestii utile pt tine**
  - Recent played: `user-read-recently-played` (poți face alte statistici). Spotify for Developers
  - Metadata de catalog (artist, track, imagini etc.) prin diverse endpointuri Web API. Spotify for Developers+2Spotify for Developers+2

Cum îl folosești pe Android:

- Faci login OAuth2 **Authorization Code cu PKCE** (flow-ul recomandat pe mobile, fără să ții client secretul pe device). Spotify for Developers
- Primești access token + refresh token → de aici poți apela Web API:
  - fie direct din aplicația Android (HTTP calls cu OkHttp/Retrofit),
  - fie printr-un backend propriu (recomandat pentru licență, îți dă și material de arhitectură / securitate în documentație).

Avantaje pentru proiectul tău:

- **Super simplu de folosit**: e doar un API REST.
- **Exact ce îți trebuie** pentru:
    - "ce ascultă acum" (pollezi endpointul o dată la N secunde),
    - top artiști / top piese (pentru profile, compatibilitate, badge-uri).
- **Portabil**: dacă mâine vrei și versiune web, tot Web API vei folosi.
- **Documentație și exemple** la greu + tutoriale actuale. Medium+1

Limitări / chestii de știut:

- Pentru control playback (seek, play, pause etc.) multe endpointuri merg **doar pentru Premium**. Spotify for Developers
    - Tu probabil nu ai nevoie să controlezi muzica, doar să o citești.
- În 2024 Spotify a început să fie mai restrictiv cu unele date de catalog pentru aplicațiile noi, deci trebuie să fii atent la ce îți permite "use case"-ul tău și termenii de utilizare (în special dacă folosești imagini, playlisturi editoriale etc.). The Verge
- Nu e "real-time la milisecundă" – tu vei face polling (ex. la 5–10 secunde) pentru harta ta, ceea ce e oricum ok pentru use-case.

2. Ce îți recomand concret

Pentru licența ta (Android app, harta cu utilizatori + ce ascultă + stats):

Folosește Spotify Web API ca bază

1. **Autentificare**
    - Implementare Authorization Code Flow with PKCE în aplicația Android → user-ul se loghează cu contul Spotify → îți dă scopurile:

        - `user-read-currently-playing`
        - `user-read-playback-state`
        - `user-top-read`
        - eventual `user-read-recently-played` pentru stats extra. [Spotify for Developers]

2. **“Ce ascultă acum”**
    - Periodic (ex. la 5–10 secunde, sau doar când user-ul deschide/activează harta):
        - chemi `GET /me/player/currently-playing` → primești track + artist + album art + progress_ms etc. [Spotify for Developers+2Spotify for Developers+2]
    - Trimiți la backend:
        - userID-ul tău intern,
        - ce ascultă (track ID, artist ID),
        - poziția pe hartă (dacă și-a dat consent pt locație),
        - timestamp.
    - Backend-ul folosește asta ca să populeze harta + filtrele de compatibilitate muzicală (exact ce ai în idei).

3. **Top artiști / top piese per utilizator**
    - `GET /me/top/artists` și `GET /me/top/tracks` cu parametrul `time_range` (short_term, medium_term, long_term) → faci profilul muzical al user-ului. [Spotify for

Developers+2spotifyweb.apidog.io+2

- Poți folosi datele astea pentru:
    - compatibilitate între utilizatori,
    - badge-uri / gamificare (“Ai descoperit X artiști noi luna asta” etc.),
    - feed personalizat.

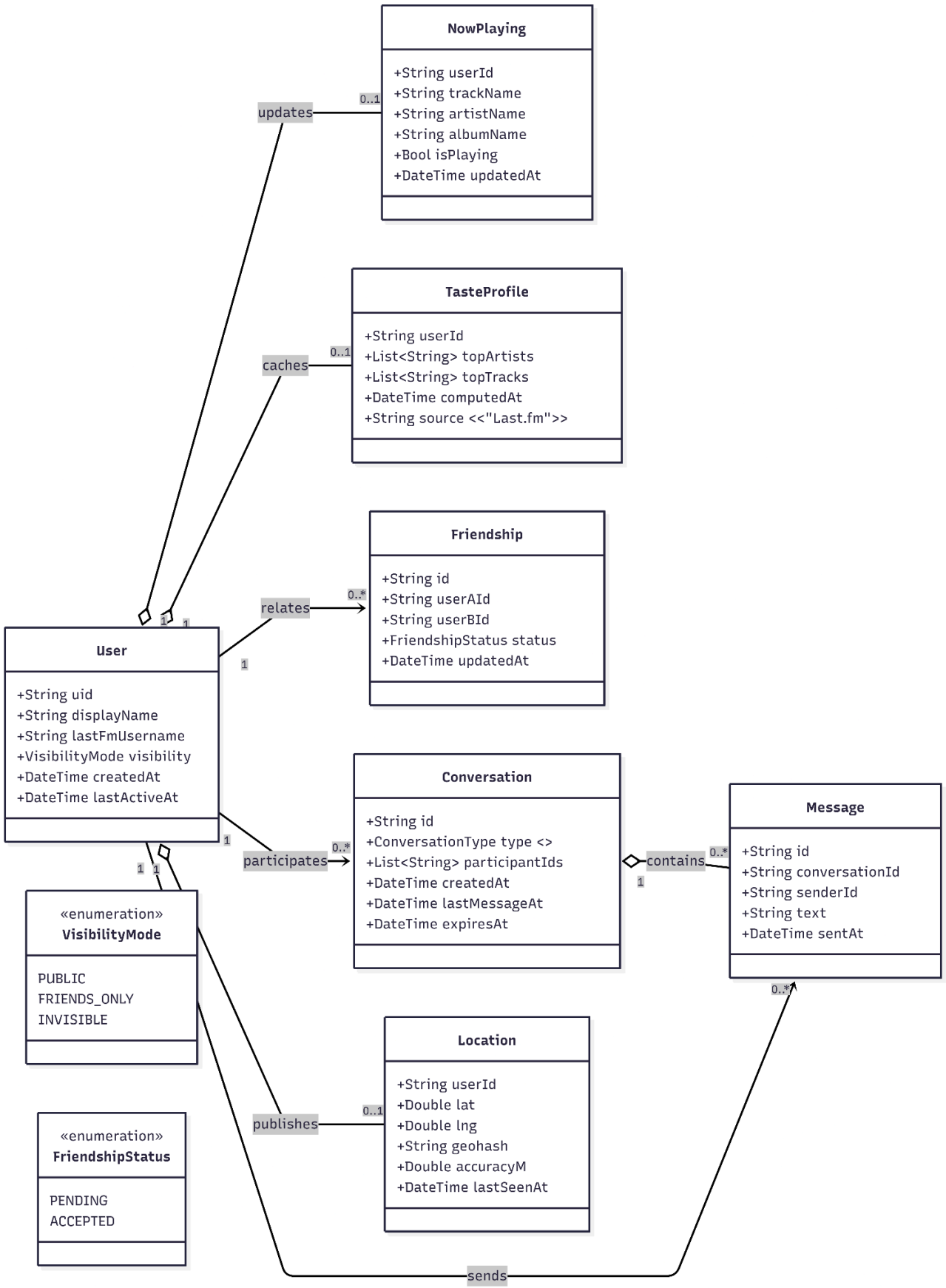
4. **Arhitectură frumoasă de licență**
    - Client Android în Kotlin (conform ce scriai în idei).
    - Un backend (ex. Spring Boot / Node / Firebase Cloud Functions) care:
        - ține userii, prieteniile, mesajele,
        - stochează snapshot-urile “ce ascultă acum”,
        - face pre-procesare pentru stats / matching.
    - Layer de “SpotifyService” care îți ambalează apelurile la Web API → poți în viitor să “plugi” și alt provider (ex. YouTube Music, dacă la un moment dat găsești un API decent).

Am decis sa merg mai departe cu last fm pentru ca spotify este momentan pus pe hold pentru developeri si asta am gasit sa fie cel mai versatil

1. Componente:
    a. UI: harta, profil, chats, settings
    b. Map API
    c. Presence publisher (ce i asta?)
    d. Music provider connector (folosesc last.fm aici)
    e. Realtime subscriptions (pentru harta, chat, notificari)
2. Firebase pentru auth, store si realtime db (sau ttl, vedem)
3. last.fm pentru chestii legate de ce asc live, top tracks sau artisti, recomandari
4. Matching service poate

Componente:

- Core (pentru chestii comune)
- Auth
- Map
- Music
- Social


NowPlaying
+String userId
+String trackName
+String artistName
+String albumName
+Bool isPlaying
+DateTime updatedAt
updates
0..1
TasteProfile
+String userId
+List<String> topArtists
+List<String> topTracks
+DateTime computedAt
+String source <<"Last.fm">>
caches
0..1
Friendship
+String id
+String userAId
+String userBId
+FriendshipStatus status
+DateTime updatedAt
relates
0..*
1
User
+String uid
+String displayName
+String lastFmUsername
+VisibilityMode visibility
+DateTime createdAt
+DateTime lastActiveAt
1
1
1
1
1
Conversation
+String id
+ConversationType type <>
+List<String> participantIds
+DateTime createdAt
+DateTime lastMessageAt
+DateTime expiresAt
participates
0..*
contains
0..*
1
Message
+String id
+String conversationId
+String senderId
+String text
+DateTime sentAt
0..*
«enumeration»
VisibilityMode
PUBLIC
FRIENDS_ONLY
INVISIBLE
Location
+String userId
+Double lat
+Double lng
+String geohash
+Double accuracyM
+DateTime lastSeenAt
publishes
0..1
«enumeration»
FriendshipStatus
PENDING
ACCEPTED
sends

1. Data modeling:
    a. User:
        i. Identification: ID, name, profile pic (?)
        ii. <Location>: lat, long, lastSeen timestamp
        iii. Privacy: public, friends only, private
        iv. Music snapshot: playingNow, title, artist, album, preview
        v. (Compatibility: top genres, top tracks, top artists (match score))
    b. Track:
        i. Title
        ii. Artist
        iii. Album
        iv. isNowPlaying
        v. previewUrl
    c. Artist:
        i. Name
        ii. Listeners?
    d. Friendship (connection between users):
        i. UserID
        ii. FriendID
        iii. Status (pending, accepted,blocked)
        iv. Match score?
    e. Location:
        i. UserID
        ii. Lat
        iii. Long
        iv. Last seen timestamp
    f. What else…
2. Music provider interface:
    a. getPlayingNow(userID): returns current track
    b. ferRecentTracks(userID): history and matching
    c. getPreviewUrl(trackID): for iTunes preview

    d. What else…

3. Map and interaction logic:
    a. Discovery radius (500m - 5km?)
    b. Filtering
    c. Zoom behavior
4. Social:
    a. “Ping”: temporary chat
    b. Friend request
    c. Matching?
    d. Chat itself
5. Technologies
    a. Front: simple kotlin compose and make prettier with figma
    b. Map engine: maplibre, map tiler, open street map
    c. Backend: Supabase (parola normala)
    d. Auth: auth din supabase
    e. DB: postgre din supabase
    f. Storage: tot din supabase
6. Logic:
    a. Now playing sync:
        i. When does it change? Pooling every n seconds or just when the song changes
        ii. If the user is on “invisible” mode, the data is fetched to their own UI, but never sent to the backend
        iii. Uses data: track, artist, …, timestamp (if user not live)
    b. Matching algo:
        i. Simple: count common artist + tracks

        ii. Advanced: assign weights, niche brings a higher score than mainstream

    c. Map interaction:
        i. Discovery: tap on user icon on the map
        ii. Preview: user can preview what other users are listening
        iii. Ping: start a temporary chat
        iv. Expiration: chat expires after 24h if users are not friends

7. Table definitions:

    a. Users:
        i. Id (PK)
        ii. Username (unique)
        iii. Email
        iv. Lastfm_username
        v. avatar_url
        vi. Privacy_mode
        vii. Created_at

    b. Location:
        i. User_id
        ii. Track_name
        iii. Artist_name
        iv. Album_art
        v. Itunes_preview_url
        vi. Is_playing
        vii. Synced_at

    c. Friendship:
        i. User_id
        ii. Friend_id
        iii. Status
        iv. match _score

8. Package structure (each with data, domain, presentation):
    a. Core (for common features)
    b. Auth
    c. Map
    d. Music

    e. Social

9. Roadmap

    a. Identity and music link
        i. Set up supabase project (auth and db)
        ii. Implement last.fm auth (one field to link last.fm username to supabase user id)

    b. Sync engine
        i. Build poolling service that calls last.fm every n seconds while app is in foreground
        ii. Implement presence publisher (update current / last track and location in db)
        iii. Implement privacy modes !!! if invisible is enabled, location is not sent to the db !!! it will appear where it was last seen

    c. Map integration
        i. Set up map libre
        ii. Fetch “nearby users” from supabase and display them (how will I mock other users?)
        iii. Implement zoom interaction (clustering)

    d. iTunes preview

        i. Fetch the preview for the specific track from iTunes API

        ii. Create the UI for the preview

    e. Pings and chat

        i. Implement the temporary chat (marked by an expires at timestamp in the conversations table)

        ii. Add friend requests -> persistent chats

        iii. Matching algo

    f. Prep for demo

        i. Create the users in a csv first

        ii. Create mock users db

iii. Simulation service and flow (accept request in db)